

稀疏一元多项式运算器 实验报告

姓名：时分

学号：PB23151799

设计目标

在C语言环境下，设计一个稀疏一元多项式运算器，要求实现如下功能：

1. 输入并创建多项式（按指数升序或降序排列，系数浮点型，指数整型）

输入浮点型系数和整型指数，建立多项式并存储。

2. 输出多项式，项数+每项系数指数（按指数升序或降序排列）

3. 求和：多项式相加

输入加数多项式的序号，输出相加结果并询问是否储存。

4. 求差：多项式相减

输入被减数与减数多项式的序号，输出相减结果并询问是否储存。

5. 求值

输入多项式序号与浮点型数值，输出多项式在该数处的取值

6. 销毁

输入多项式序号，删除多项式全部内容

7. 清空

输入多项式序号，将多项式清零

8. 修改

1. 插入新项：输入多项式序号、新项的系数与指数，将新项插入多项式

2. 删除项：输入多项式序号、需删除项的指数，将该项从多项式中删除

3. 修改已有项：输入多项式序号、需修改项的指数、修改后的系数与指数，修改该项

9. 微分（N阶导数）

输入多项式序号和需求的微分阶数，输出微分结果并询问是否储存

10. 不定积分

输入多项式序号，输出不定积分结果并询问是否储存

11. 定积分

输入多项式序号与定积分起点、终点，输出定积分结果

12. 乘法和乘方

1. 乘法：输入乘数多项式的序号，输出相乘结果并询问是否储存

2. 乘方：输入乘方多项式的序号与乘方次数，输出乘方结果并询问是否储存

13. 带余除法

输入被除数与除数多项式的序号，输出商多项式与余数多项式，分别询问是否储存

算法设计

数据结构

采取菜单式驱动，循环输出菜单界面，输入操作序号，执行对应操作。

采用带头结点的链表存储多项式，链表类型定义如下：

```
typedef struct LNode{
    double coef; //该项系数
    int exp;      //该项指数
    struct LNode *next;
}LNode,*LinkList;
```

使用链表头结点的exp变量存储链表长度。

采用链表指针数组 p[NUM] 存储各链表的头指针，数组元素初值为NULL，创建链表时从头开始查找空位置，将链表头指针存入第一个空位置中。

除法功能结果较为特殊，包含商多项式和余数多项式，故采用两个链表头结点组成的结构体类型存储除法的结果，类型定义如下：

```
typedef struct DivResult{
    LinkList quotient; //商多项式的头指针
    LinkList remainder; //余数多项式的头指针
}DivResult;
```

程序结构

各函数原型、功能和接口的描述如下：

```
int Locate(LinkList L,int nexp);
    //查找多项式内是否有指定的指数，有则返回该项的位置，没有返回0
Status InitList(LinkList &L);
    //在指定位置创建系数从高到低的多项式，包括输入功能，输入-1结束输入
Status InitList(LinkList L);
    //输出指定多项式
LinkList AddList(LinkList La,LinkList Lb);
    //创建一个新多项式，内容为两个输入的多项式相加，返回结果的头指针
LinkList SubList(LinkList La,LinkList Lb);
    //创建一个新多项式，内容为两个输入的多项式相减，返回结果的头指针
double EvaluateList(LinkList L,double x);
    //代入x的值计算多项式的值并返回
Status DestroyList(LinkList &L);
    //释放链表L包括头结点内的所有结点
Status ClearList(LinkList &L);
    //释放链表L头结点外的所有结点
Status InsertNode(LinkList &L,int nexp,double ncoe);
    //在链表L中按顺序插入新结点
Status DeleteNode(LinkList &L,int location);
    //删除链表L指定位置的结点
Status EditNode(LinkList &L,int location,int nexp,double ncoe);
    //修改链表L指定位置的结点
LinkList DiffList(LinkList L);
    //计算L的一阶导数存入新链表，返回新链表的头指针
LinkList InteList(LinkList L);
    //计算L的不定积分（常数项为0）存入新链表，返回新链表的头指针
LinkList MultList(LinkList La,LinkList Lb);
    //创建一个新多项式，内容为两个输入的多项式相乘，返回结果的头指针
DivResult DivList(LinkList La,LinkList Lb);
```

```
//输入被除式与除式，以结构体形式返回商和余数的头指针
int menu(void);
//打印菜单界面，输入操作序号并返回
```

调试分析

各功能测试数据与问题解决如下：

1. 输入并创建多项式：

1. 常规输入 1 2.5 3 6.1 8 -2.3 -1，没有出现问题。

2. 输入负指数 -2 1 -1，问题：创建了含负指数结点。

解决方法：在输入指数时添加合法性检测，只允许输入非负整数和-1（结束指令）。
为插入、修改同样加入该检测。

3. 输入重复指数 1 1 1 1 -1，问题：创建了指数重复的结点。

解决方法：在输入指数后利用Locate函数检测是否重复，若重复则提示重新输入。
为插入、修改同样加入该检测。

4. 输入零系数 2 0 -1，问题：创建了零系数结点，空占位置。

解决方法：在输入系数时检测是否为0，若为0则不创建结点。
为插入、修改同样加入该检测。

2. 输出多项式：

1. 常规输入，没有出现问题。

2. 输入未创建的序号，问题：无法输出未创建的多项式。

解决方法：为各功能输入序号时添加合法性检测，若当前位置元素为NULL，则提示未创建多项式。

3. 输入仅含头结点多项式的序号，问题：无法输出仅含头结点的多项式。

解决方法：在输出前增加判断，若仅含头结点，则输出0.000。

3. 求和：

1. 常规输入，没有出现问题。

2. 输入含有指数相同、系数互为相反数项的多项式，问题：结果出现零系数结点。

解决方法：计算同指数项时检测系数结果是否为0，若为0则不创建结点。
在求差中同样加入该检测。

4. 求差：没有出现问题。

5. 求值：没有出现问题。

6. 销毁：注意将数组p对应位置重置为NULL，没有出现问题。

7. 清空：没有出现问题。

8. 插入新项：已添加合法性检测，没有出现问题。

9. 删除项：利用Locate函数寻找删除项位置，该项不存在则打印未找到该项，没有出现问题。

10. 修改已有项：已添加合法性检测，没有出现问题。

11. 微分：

1. 常规输入，没有出现问题。

2. 输入负数阶数，问题：无法计算负阶数微分。

解决方法：加入合法性检测。

在乘方中同样加入该检测。

12. 不定积分：没有出现问题。

13. 定积分：没有出现问题。

14. 乘法：没有出现问题。

15. 乘方：没有出现问题。

16. 除法：

1. 常规输入，问题：商多项式项数错误。

解决方法：在计算过程中添加商多项式 $L \rightarrow \text{exp}$ 的自增。

2. 被除数输入0，问题：结果错误。

解决方法：加入合法性检测。

3. 除数项数大于被除数项数，问题：结果错误。

解决方法：加入合法性检测。

算法的时空分析

各函数时空复杂度如下：（L长度为n，La长度为a，Lb长度为b）

```
int Locate(LinkList L,int nexp);
//查找多项式内是否有指定的指数，有则返回该项的位置，没有返回0
//时间复杂度：O(n)
//空间复杂度：O(1)

Status InitList(LinkList &L);
//在指定位置创建系数从高到低的多项式，包括输入功能，输入-1结束输入
//时间复杂度：O(n^2)
//空间复杂度：O(n)

Status InitList(LinkList L);
//输出指定多项式
//时间复杂度：O(n)
//空间复杂度：O(1)

LinkList AddList(LinkList La,LinkList Lb);
//创建一个新多项式，内容为两个输入的多项式相加，返回结果的头指针
//时间复杂度：O(a+b)
//空间复杂度：O(a+b)

LinkList SubList(LinkList La,LinkList Lb);
//创建一个新多项式，内容为两个输入的多项式相减，返回结果的头指针
//时间复杂度：O(a+b)
//空间复杂度：O(a+b)

double EvaluateList(LinkList L,double x);
//代入x的值计算多项式的值并返回
//时间复杂度：O(n)
//空间复杂度：O(1)

Status DestroyList(LinkList &L);
//释放链表L包括头结点内的所有结点
//时间复杂度：O(n)
//空间复杂度：O(1)

Status ClearList(LinkList &L);
//释放链表L头结点外的所有结点
//时间复杂度：O(n)
//空间复杂度：O(1)

Status InsertNode(LinkList &L,int nexp,double ncoe);
//在链表L中按顺序插入新结点
```

```

//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 
Status DeleteNode(LinkList &L,int location);
//删除链表L指定位置的结点
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 
Status EditNode(LinkList &L,int location,int next,double ncoe);
//修改链表L指定位置的结点
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 
LinkList DiffList(LinkList L);
//计算L的一阶导数存入新链表,返回新链表的头指针
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(n)$ 
LinkList InteList(LinkList L);
//计算L的不定积分(常数项为0)存入新链表,返回新链表的头指针
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(n)$ 
LinkList MultList(LinkList La,LinkList Lb);
//创建一个新多项式,内容为两个输入的多项式相乘,返回结果的头指针
//时间复杂度:  $O(a*b)$ 
//空间复杂度:  $O(a*b)$ 
DivResult DivList(LinkList La,LinkList Lb);
//输入被除式与除式,以结构体形式输出商和余数的头指针
//时间复杂度:  $O(a*b)$ 
//空间复杂度:  $O(3a)$ 

```

测试结果及分析

-----多项式计算器-----

- 1、输入并创建多项式
- 2、输出多项式
- 3、求和
- 4、求差
- 5、求值
- 6、销毁
- 7、清空
- 8、修改
- 9、微分
- 10、不定积分
- 11、定积分
- 12、乘法和乘方
- 13、除法
- 0、退出程序

(之后省略菜单)

请输入操作: 1

请输入指数(输入-1结束输入): 1

请输入系数: -2

请输入指数(输入-1结束输入): 0

请输入系数: 5

请输入指数(输入-1结束输入): 3

请输入系数: 1.4

请输入指数(输入-1结束输入): -1

创建成功,多项式编号为1,按回车键返回菜单

请输入操作：1

请输入指数（输入-1结束输入）：-1

//创建多项式为0

创建成功，多项式编号为2，按回车键返回菜单

请输入操作：1

请输入指数（输入-1结束输入）：1

请输入系数：2

请输入指数（输入-1结束输入）：4

请输入系数：-2.3

请输入指数（输入-1结束输入）：-1

创建成功，多项式编号为3，按回车键返回菜单

请输入操作：2

请输入多项式编号：1

项数为：3

多项式为：

1.400 x^3 -2.000 x +5.000

输出完毕，按回车键返回菜单

请输入操作：3

请输入需要相加的第一个多项式编号：1

请输入需要相加的第二个多项式编号：3

项数为：3

多项式为：

-2.300 x^4 +1.400 x^3 +5.000

要存储该结果吗？（Y/N）：Y

存储成功，多项式编号为4，按回车键返回菜单

请输入操作：4

请输入被减数多项式编号：4

请输入减数多项式编号：3

项数为：3

多项式为：

1.400 x^3 -2.000 x +5.000

要存储该结果吗？（Y/N）：N

计算完毕，按回车键返回菜单

请输入操作：5

请输入多项式编号：1

请输入x的值：1

代入计算结果为 4.400，按回车键返回菜单

请输入操作：7

请输入多项式编号：4

清空完毕，按回车键返回菜单

请输入操作：2

请输入多项式编号：4

多项式为：0.000

//仅含头结点多项式为0

输出完毕，按回车键返回菜单

请输入操作：6

请输入多项式编号：4

销毁完毕，按回车键返回菜单

请输入操作：2

请输入多项式编号：4

该编号下无多项式！

//未找到多项式，说明正常销毁

请输入操作：8

请输入多项式编号：1

请选择操作：

1、插入新的结点

2、删除已有结点

3、修改已有结点的系数和指数

0、退出

(之后省略菜单)

1

请输入新结点的指数（输入-1取消）：5

请输入新结点的系数：2

插入成功！当前项数为：4

多项式为：

2.000 x^5 +1.400 x^3 -2.000 x +5.000

按回车键继续

请输入操作：8

请输入多项式编号：1

请选择操作：

2

请输入删除结点的指数（输入-1取消）：5

删除成功！当前项数为：3

多项式为：

1.400 x^3 -2.000 x +5.000

按回车键继续

请输入操作：8

请输入多项式编号：1

请选择操作：

3

请输入需修改结点的指数（输入-1取消）：4

未找到该项，请重新输入！

请输入需修改结点的指数（输入-1取消）：3

请输入新指数：2

请输入新系数：0.8

修改成功！当前项数为：3

多项式为：

0.800 x^2 -2.000 x +5.000

按回车键继续

请输入操作：9

请输入需要微分的多项式编号：1

请输入需求的微分阶数：2

项数为：1

多项式为：

1.600 //结果正确

要存储该结果吗？（Y/N）：N

计算完毕，按回车键返回菜单

请输入操作：10

请输入需要求不定积分的多项式编号：1

项数为：3

多项式为：

0.267 x^3 -1.000 x^2 +5.000 x +C （C为任意常数） //结果正确

要存储该结果吗？（Y/N）：N

计算完毕，按回车键返回菜单

```
请输入操作：11
请输入需要定积分的多项式编号：1
请输入定积分起点：2
请输入定积分终点：4
定积分结果为：12.933 //结果正确
计算完毕，按回车键返回菜单

请输入操作：12
1. 多项式相乘
2. 多项式乘方
请输入操作：1
请输入需要相乘的第一个多项式编号：1
请输入需要相乘的第二个多项式编号：3
项数为：6
多项式为：
-1.840 x^6 +4.600 x^5 -11.500 x^4 +1.600 x^3 -4.000 x^2 +10.000 x //结果正确
要存储该结果吗？（Y/N）：Y
存储成功，多项式编号为4，按回车键返回菜单

请输入操作：12
1. 多项式相乘
2. 多项式乘方
请输入操作：2
请输入需要乘方的多项式编号：1
请输入乘方次数：2
项数为：5
多项式为：
0.640 x^4 -3.200 x^3 +12.000 x^2 -20.000 x +25.000 //结果正确
要存储该结果吗？（Y/N）：N
计算完毕，按回车键返回菜单

请输入操作：13
请输入被除数多项式编号：4
请输入除数多项式编号：2
除数为0

请输入操作：13
请输入被除数多项式编号：4
请输入除数多项式编号：3
商项数为：3
多项式为：
0.800 x^2 -2.000 x +5.000 //结果正确
余数多项式为：0.000 //结果正确
要存储商吗？（Y/N）：N
要存储余数吗？（Y/N）：N
计算完毕，按回车键返回菜单

请输入操作：0
感谢您的使用，再会！

-----
Process exited after 342.8 seconds with return value 1
请按任意键继续。 . . .
```

实验体会与收获

通过本实验，我复习巩固了指针和链表操作的基本功和数组操作的基本功，进一步学习了模块化程序设计的风格，开始注重人机交互界面设计和源程序的整洁性、可读性，更重视对程序健壮性的处理与输入的合法性检测，训练了程序的调试技术，开始注重程序的时空效率，为未来设计高效率算法和编写复杂程序打下基础。